GPUMODE Lecture Study Notes: Lecture 12 – Flash Attention

Core Problem the Lecture Addresses

This lecture explains the implementation logic behind **Flash Attention**, an IO-aware GPU algorithm for computing attention without materializing the full attention matrix in high-bandwidth memory.

The standard attention computation is

$$P = \text{softmax}\left(QK^T \cdot \frac{1}{\sqrt{d}}\right), \qquad O = PV.$$

Here, Q, K, and V are the query, key, and value matrices; d is the head dimension; P is the attention probability matrix; and O is the output.

The expensive intermediate is

$$P \in \mathbb{R}^{N \times N},$$

where N is the sequence length. Standard attention computes QK^T, writes logits or probabilities to HBM, reads them back, and then multiplies by V. Flash Attention avoids this by tiling the computation and keeping intermediate state on-chip.

> Flash Attention is not a new attention formula. It is a different execution schedule for the same formula, designed to reduce HBM traffic and exploit shared memory and registers.

Required Background

Attention as a Classification-Like Operation

The speaker frames attention as similar to classification. In a conventional classifier, an input activation

$$x \in \mathbb{R}^d$$

is multiplied by a weight matrix

$$W \in \mathbb{R}^{C \times d}$$

to produce class logits

$$z = Wx,$$

followed by

$$p = \mathrm{softmax}(z).$$

Each row W_c can be viewed as a class embedding. The class logit is a dot product:

$$z_c = \langle W_c, x \rangle.$$

Attention has the same flavor. A query vector Q_t compares itself against key vectors K_s. The resulting dot products act like logits over sequence positions:

$$S_{t,s} = \frac{1}{\sqrt{d}} \langle Q_t, K_s \rangle.$$

The softmax turns those logits into probabilities over value positions:

$$P_{t,s} = \frac{\exp(S_{t,s})}{\sum_{r=1}^{N} \exp(S_{t,r})}.$$

Then the output is a weighted sum of values:

$$O_t = \sum_{s=1}^{N} P_{t,s} V_s.$$

Thus attention asks: for this query position, which value rows should be selected, mixed, or emphasized?

Multi-Head Attention

In multi-head attention, the model performs several independent attention computations. If the model dimension is d_{model} and there are H heads, each head often has dimension

$$d_h = \frac{d_{\text{model}}}{H}.$$

Each head computes

$$O^{(h)} = \text{softmax}\left(Q^{(h)}{K^{(h)}}^T \cdot \frac{1}{\sqrt{d_h}}\right) V^{(h)}.$$

The lecture emphasizes that this is useful not only representationally but also computationally. Heads are independent, so different heads and batch elements provide many independent units of GPU work.

CUDA Thread Blocks and SM Mapping

CUDA kernels are launched as a grid of thread blocks. A thread block is a group of cooperating threads that execute on a single streaming multiprocessor, or SM.

A simplified mapping used in the lecture is

$$\text{one attention head} \longrightarrow \text{one CUDA block} \longrightarrow \text{one SM at a time.}$$

This mapping matters because threads within a block can share data through shared memory. Since softmax has row-wise dependencies, the threads working on one head need cooperation.

A block-level implementation can use:

- shared memory for Q, K, and V tiles,
- registers for local accumulators,
- synchronization among threads,
- and one final global write of O.

Main Concepts

Standard Attention Dataflow

For one head, assume

$$Q, K, V \in \mathbb{R}^{N \times d}.$$

The standard implementation performs

$$S = QK^T,$$

$$P = \mathrm{softmax}(S),$$

$$O = PV.$$

The shapes are

$$S \in \mathbb{R}^{N \times N}, \qquad P \in \mathbb{R}^{N \times N}, \qquad O \in \mathbb{R}^{N \times d}.$$

The memory problem is that S and P are quadratic in sequence length. For example, if $N = 4096$, one attention matrix has

$$4096^2 = 16{,}777{,}216$$

entries. In FP16, this is approximately

$$16{,}777{,}216 \times 2 = 33{,}554{,}432 \text{ bytes} \approx 32 \text{ MiB}$$

per head.

Flash Attention Dataflow

Flash Attention avoids writing the full S or P matrices to HBM. Instead, it tiles the computation.

A simplified tile-level structure is:

1. Load a tile of Q into on-chip memory.
2. Initialize row-wise softmax statistics.
3. Loop over tiles of K and V.
4. Compute local logits for the current Q-tile and K-tile.
5. Update the online softmax maximum and denominator.
6. Update the output accumulator using the current V-tile.
7. After all K, V tiles are processed, normalize and write O.

The key principle is

$$\text{read } Q, K, V \text{ tiles} \rightarrow \text{compute on-chip} \rightarrow \text{write only } O.$$

The Crucial Index Coincidence

The output element is

$$O_{t,j} = \sum_{s=1}^{N} P_{t,s} V_{s,j},$$

where

$$P_{t,s} = \frac{\exp(S_{t,s})}{\sum_{r=1}^{N} \exp(S_{t,r})},$$

and

$$S_{t,s} = \frac{1}{\sqrt{d}} \sum_{k=1}^{d} Q_{t,k} K_{s,k}.$$

The important observation is that the softmax dimension and the PV contraction dimension are both the source sequence dimension s:

$$\text{softmax reduction dimension} = s, \qquad \text{value accumulation dimension} = s.$$

This alignment is the mathematical reason Flash Attention can fuse the two matrix multiplications with the softmax.

Tiling Strategy

Let B_t be the number of query rows in a tile and B_s be the number of key/value rows in a tile.

The kernel loads

$$Q_{\text{tile}} \in \mathbb{R}^{B_t \times d},$$

$$K_{\text{tile}} \in \mathbb{R}^{B_s \times d},$$

$$V_{\text{tile}} \in \mathbb{R}^{B_s \times d}.$$

Then it computes local logits:

$$S_{\text{tile}} = Q_{\text{tile}} K_{\text{tile}}^{T}, \qquad S_{\text{tile}} \in \mathbb{R}^{B_t \times B_s}.$$

The kernel does not store S_{tile} globally. It uses it immediately to update softmax statistics and output accumulators.

Hardware-Level Explanation

HBM Versus Shared Memory Versus Registers

Flash Attention is an IO-aware algorithm. The performance win comes from exploiting the GPU memory hierarchy:

- **HBM / DRAM**: large but expensive to access.
- **L2 cache**: shared across SMs.
- **L1 cache / shared memory**: on-chip and much faster than HBM.
- **Registers**: fastest storage, private to each thread.

Writing intermediates to DRAM and reading them back is expensive. If data can stay in shared memory or registers, the kernel avoids large memory traffic.

Why One Head Per Block Is a Natural Starting Point

A single attention head has substantial internal dependency because softmax over one query row requires all key positions. If multiple blocks independently compute pieces of the same softmax row, they need cross-block communication or extra reduction passes.

Mapping one head, or one head tile, to one block allows threads to cooperate using shared memory:

$$\text{thread block} = \text{cooperating threads}.$$

$$\text{cooperating threads} \Rightarrow \text{shared memory and synchronization}.$$

This is why the head dimension becomes a limiting factor. A larger head dimension increases shared memory usage, register usage, arithmetic per dot product, and compiler scheduling pressure.

Shared Memory Usage

If Q, K, and V tiles are staged in shared memory, the approximate memory usage in FP32 is

$$4B_t d + 4B_s d + 4B_s d,$$

or

$$4d(B_t + 2B_s) \text{ bytes.}$$

For $B_t = 64$, $B_s = 64$, and $d = 128$,

$$4 \cdot 128 \cdot (64 + 128) = 98{,}304 \text{ bytes.}$$

This is a large amount of shared memory for one block.

Why Registers Matter

The lecture discusses moving output accumulators from shared memory into registers. This can dramatically speed up an educational kernel.

A small output fragment can be accumulated as local variables:

$$O_{\text{frag}}[0], \quad O_{\text{frag}}[1], \quad \ldots, \quad O_{\text{frag}}[r-1].$$

Registers are private to each thread and much faster than shared memory. However, registers are limited. If the compiler cannot fit all live variables into registers, it spills them to local memory. Local memory is backed by global memory and cache, so spilling can be very expensive.

Register Pressure and Occupancy

A simple resource model is

$$R_{\text{SM}} \geq R_{\text{thread}} \times T_{\text{block}} \times B_{\text{resident}},$$

where R_{SM} is the register file capacity per SM, R_{thread} is registers per thread, T_{block} is threads per block, and B_{resident} is resident blocks per SM.

If R_{thread} is too high, occupancy decreases. If the compiler cannot allocate enough registers, spilling occurs.

The lecture mentions practical ways to detect this:

- inspect compiler output,
- check PTX or SASS,
- use Compiler Explorer,
- use Nsight Compute,
- look for local memory accesses,
- check compiler register usage reports.

Mathematical Reasoning

Naive Softmax

For logits x_i, softmax is

$$p_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}.$$

The issue is that $\exp(x_i)$ can overflow for large x_i. For example, $\exp(100)$ is already extremely large in floating-point arithmetic.

Stable Softmax

To stabilize softmax, subtract the maximum:

$$m = \max_j x_j.$$

Then

$$p_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}.$$

This is mathematically equivalent:

$$\begin{aligned} p_i &= \frac{\exp(x_i)}{\sum_j \exp(x_j)} && (1) \\ &= \frac{\exp(x_i)\exp(-m)}{\sum_j \exp(x_j)\exp(-m)} && (2) \\ &= \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}. && (3) \end{aligned}$$

Now the largest exponent is

$$\exp(m - m) = 1,$$

and all other exponentials are at most 1.

Online Softmax

Flash Attention processes logits in blocks, so it cannot compute the full row maximum and denominator in one step. Instead, it maintains running values.

Let m_{old} be the old maximum and ℓ_{old} be the old denominator-like sum:

$$\ell_{\text{old}} = \sum_{x \in \text{old blocks}} \exp(x - m_{\text{old}}).$$

For a new block, compute

$$m_{\text{block}} = \max_{x \in \text{new block}} x.$$

Then update the running maximum:

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{block}}).$$

The old denominator must be rescaled to the new maximum:

$$\ell_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}})\ell_{\text{old}} + \sum_{x \in \text{new block}} \exp(x - m_{\text{new}}).$$

This formula is the core of online softmax.

Online Output Accumulation

The output accumulator must be rescaled whenever the softmax maximum changes.

Let O_{old} be the old unnormalized output accumulator:

$$O_{\text{old}} = \sum_{x \in \text{old blocks}} \exp(x - m_{\text{old}})V_x.$$

After seeing a new block, update:

$$O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}})O_{\text{old}} + \sum_{x \in \text{new block}} \exp(x - m_{\text{new}})V_x.$$

At the end, normalize:

$$O = \frac{O_{\text{final}}}{\ell_{\text{final}}}.$$

This is the key trick that allows Flash Attention to avoid storing P.

Code Walkthrough

Naive PyTorch Attention

```
import torch

def naive_attention(q, k, v):
    d = q.shape[-1]
    scale = 1.0 / (d ** 0.5)

    s = q @ k.transpose(-1, -2)
    p = torch.softmax(s * scale, dim=-1)
    o = p @ v

    return o
```

This is mathematically clear but memory inefficient. The tensor s is the full logits matrix, and p is the full attention probability matrix. These are exactly the intermediates Flash Attention avoids writing to HBM.

Flash Attention-Like Pseudocode

```
def flash_attention_reference(Q, K, V, block_m, block_n):
    N, d = Q.shape
    O = zeros_like(Q)

    for q_start in range(0, N, block_m):
        q_end = min(q_start + block_m, N)
        Q_tile = Q[q_start:q_end, :]

        m = full((q_end - q_start,), -inf)
        l = zeros((q_end - q_start,))
        O_acc = zeros((q_end - q_start, d))

        for kv_start in range(0, N, block_n):
            kv_end = min(kv_start + block_n, N)
            K_tile = K[kv_start:kv_end, :]
            V_tile = V[kv_start:kv_end, :]

            S = Q_tile @ K_tile.T

            m_block = rowmax(S)
            m_new = maximum(m, m_block)

            P_unnorm = exp(S - m_new[:, None])
            old_scale = exp(m - m_new)
```

```
            l = old_scale * l + rowsum(P_unnorm)
            O_acc = old_scale[:, None] * O_acc + P_unnorm @ V_tile

            m = m_new

        O[q_start:q_end, :] = O_acc / l[:, None]

    return O
```

The important pieces are:

- m tracks the running row maximum.
- ℓ tracks the running softmax denominator.
- O_{acc} tracks the unnormalized output.
- O_{acc} is rescaled whenever m changes.
- The full P matrix is never stored.

CUDA-Style Kernel Skeleton

```
extern "C" __global__
void flash_attention_forward(
    const float* __restrict__ Q,
    const float* __restrict__ K,
    const float* __restrict__ V,
    float* __restrict__ O,
    int N,
    int D
) {
    extern __shared__ float smem[];

    float* Q_tile = smem;
    float* K_tile = Q_tile + BLOCK_M * D;
    float* V_tile = K_tile + BLOCK_N * D;

    int tid = threadIdx.x;
    int block_row = blockIdx.x;
    int q_start = block_row * BLOCK_M;

    float m_frag = -INFINITY;
    float l_frag = 0.0f;
    float o_frag[FRAG_D];

    #pragma unroll
    for (int j = 0; j < FRAG_D; ++j) {
        o_frag[j] = 0.0f;
```

```
    }

    for (int idx = tid; idx < BLOCK_M * D; idx += blockDim.x) {
        Q_tile[idx] = Q[q_start * D + idx];
    }

    __syncthreads();

    for (int kv_start = 0; kv_start < N; kv_start += BLOCK_N) {

        for (int idx = tid; idx < BLOCK_N * D; idx += blockDim.x) {
            K_tile[idx] = K[kv_start * D + idx];
            V_tile[idx] = V[kv_start * D + idx];
        }

        __syncthreads();

        /*
        Compute local logits:
            S_tile = Q_tile @ K_tile.T

        Update:
            m_new
            l_new
            o_frag

        Keep partial output in registers when possible.
        */

        __syncthreads();
    }

    /*
    Normalize:
        O = o_frag / l_frag

    Store final output to global memory.
    */
}
```

This skeleton shows the central memory movement pattern:

Q, K, V tiles $\rightarrow$ shared memory $\rightarrow$ register accumulators $\rightarrow$ O in HBM.

Performance Intuition

Why Flash Attention Is Faster

The main improvement is reducing HBM traffic.

Naive attention has memory movement roughly like

$$Q, K, V \to S \to P \to O,$$

where S and P are $N \times N$.

Flash Attention has memory movement closer to

$$Q, K, V \to \text{on-chip tiled computation} \to O.$$

The compute complexity is still

$$O(N^2 d),$$

but the memory footprint for intermediates is reduced from quadratic storage to tile-sized on-chip storage.

Arithmetic Intensity

Arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

Flash Attention improves arithmetic intensity because each loaded tile participates in more useful compute before being evicted.

Why Automatic Compiler Fusion Is Not Enough

Compiler fusion helps with memory-bound elementwise operations, but Flash Attention is more complex. A compiler would need to rediscover:

- the tiling strategy,
- online softmax,
- output rescaling,
- shared memory layout,
- register allocation,
- and tensor core usage.

Flash Attention is not merely adjacent operator fusion. It is an algorithmic rescheduling of attention around the GPU memory hierarchy.

Why Causal Masks Complicate Tiling

Causal attention requires

$$S_{t,s} = -\infty \quad \text{when} \quad s > t.$$

This creates a triangular valid region. In tile space, some blocks are fully valid, some are fully invalid, and some are partially valid. A production kernel must avoid wasting work on invalid tiles while still computing correct softmax statistics for partially valid tiles.

Common Misconceptions

Misconception: Flash Attention Approximates Attention

Flash Attention computes the same mathematical attention expression, up to floating-point ordering differences:

$$O = \text{softmax}\left(QK^T \cdot \frac{1}{\sqrt{d}}\right) V.$$

It is not an approximation method.

Misconception: Softmax Is Just an Elementwise Function

Softmax is not elementwise. It requires a row-wise reduction:

$$\sum_j \exp(x_j).$$

This is why tiled softmax requires online maximum and denominator tracking.

Misconception: Shared Memory Solves Everything

Shared memory is fast but small. Flash Attention quickly hits shared memory limits when storing Q, K, V, logits, and accumulators. This is why practical kernels rely heavily on registers and carefully chosen tile sizes.

Misconception: More Fusion Is Always Better

More fusion can reduce memory traffic, but it increases register pressure, shared memory pressure, instruction count, compile time, scheduling complexity, and risk of spilling.

Practical Takeaways

- Flash Attention is best understood as **IO-aware exact attention**.
- The algorithm avoids materializing the $N \times N$ attention matrix.
- The mathematical trick is online softmax.
- The hardware trick is tiling into shared memory and accumulating in registers.
- The softmax dimension and value contraction dimension both use the sequence index s, enabling fusion.
- Head dimension is a major practical constraint because it controls shared memory and register pressure.
- Production Flash Attention is difficult because it must combine tensor cores, masks, dropout, backward pass, and architecture-specific tuning.

Project or Experiment Ideas

1. Implement naive attention in PyTorch and measure memory usage as N increases.
2. Implement online softmax for a single vector and verify it matches stable softmax.
3. Implement a tiled attention reference in Python using explicit loops.
4. Compare the tiled reference against PyTorch scaled dot-product attention.
5. Write a Numba CUDA kernel for single-head attention.
6. Move output accumulators from shared memory to registers and benchmark the effect.
7. Use Nsight Compute to inspect register usage, occupancy, shared memory bandwidth, and local memory spills.
8. Add causal masking and observe how triangular tile regions affect performance.
9. Experiment with tile sizes 32, 64, and 128.

Final Mental Model

Standard attention says:

> Build the entire $N \times N$ attention matrix, store it, normalize it, read it back, and then multiply by values.

Flash Attention says:

For a tile of queries, stream through keys and values, maintain softmax statistics online, accumulate the output directly, and write only the final result.

The central idea is

Do not move large intermediates through HBM if the GPU can compute with them on-chip.

Flash Attention is therefore one of the clearest examples of modern deep learning systems work: the math is familiar, but the performance comes from changing the memory schedule to match GPU hardware.

Visual Concept

Draw two pipelines side by side.

- **Standard attention:** show Q and K being multiplied to form

  $$S = QK^T.$$

  Then show S written to HBM. Next show

  $$P = \mathrm{softmax}(S).$$

  Also show P written to HBM. Finally, show P being read back and multiplied with V:

  $$O = PV.$$

- **Flash Attention:** show small tiles of Q, K, and V loaded into shared memory. Inside the SM, show running updates of

  $$m, \qquad \ell, \qquad O_{\mathrm{acc}}.$$

  Then show only the final output O being written back to HBM.